Software Requirements Specification (SRS)

Project Title: Movie Explorer - ReactJS Web Application

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the complete structural, functional, and non-functional requirements of the **Movie Explorer Web Application**. This application is designed to be an industry-standard, component-based Single Page Application (SPA) built using ReactJS that interfaces with external web services via The Movie Database (TMDB) API platform. This document acts as an explicit technical contract and engineering blueprint for frontend UI/UX development, component reuse, routing configurations, and asynchronous state integration.

1.2 Scope

The web platform centers on delivering a highly interactive, responsive movie exploration interface. The scope spans across the implementation of client-side logic to handle:

- **Catalog Browsing Feeds:** Dynamically rendering separate rows for Trending, Popular, Top Rated, and Upcoming movies.
- **Real-time Title Query Engine:** Text-based instant lookups filtering down matching movie assets.
- **Deep Dynamic Item Routing:** Parameterized paths loading deep cinematic properties including layout posters, category genres, durations, calendar dates, and viewer ratings.
- **Cast & Relation Links:** Displaying performance credit lines alongside similar genre recommendation streams.
- **Local Memory Persistence:** A personalized user watchlist saving and deleting selections across runtime sessions utilizing browser-localized engine caches without active login dependencies.
- **Data Pagination and Sorting:** Explicit drop-down filters combined with multi-page offset indexing directly tied into the network requests lifecycle.

1.3 Definitions

Term	Meaning
SPA	Single Page Application; a web framework loading a single HTML container that rewrites sections dynamically as user click events occur.
TMDB API	The Movie Database API; a third-party RESTful web service providing global movie metadata frames.

Term	Meaning
Axios Instance	A customized HTTP communication client configured with unique base URLs and permanent parameter injectors.
Context API	A React framework engine used to manage state changes globally down an nested component tree without prop-drilling errors.
LocalStorage	A native browser key-value string allocation space that keeps data cache safe across manual window refreshes.

1.4 References

- Provided Frontend Track Specification File: FEWT_Project_Description.docx.
- TMDB Developer API v3 Core Integration Guidelines.

2. Overall Description

2.1 Product Perspective

The Movie Explorer web application operates as a standalone frontend presentation client that delegates its entire data sourcing load asynchronously to the third-party TMDB cloud infrastructure.

Frontend Isolated System Stack:

- **View Framework Layer:** ReactJS utilizing functional components and hooks architectures.
- **Client-Side Routing Engine:** React Router (v6) mapping nested states and deep dynamic query string keys.
- **Asynchronous Transport Protocol:** Axios configurations incorporating centralized security environment injections.
- **Global State Orchestration:** React Context API providing unified access scopes for localized lists tracking.
- **Presentation Design Grid:** Material UI or Bootstrap layout utility libraries compiling fluid grid components.
- **Hosting Pipeline target:** Netlify, GitHub Pages, or Vercel production compilation builds.

2.2 User Classes

The frontend client implements an open-access platform structure prioritizing fast navigation lookup states:

User Role	Client Interface System Privileges
Guest User	Unrestricted open access to browse real-time feeds, query search modules, look up cast credits, sort records via filter criteria, and maintain an updated collection within the local browser watchlist without registering accounts .

2.3 Operating Environment

- **Target Web Browsers:** Google Chrome (v100+), Apple Safari (v15+), Mozilla Firefox (v100+), Microsoft Edge.
- **Device Responsiveness Requirements:** Fluid breakpoints transforming content cards uniformly across Mobile phones, Tablet platforms, and wide Desktop screens .
- **Runtime Constraints:** Requires client-side network connections to perform real-time HTTP transport executions targeting TMDB web server domains.

3. Client-Side State Schema Definitions (Data Shapes)

To prevent component rendering breakdowns and ensure clean data mapping, students must configure local component states, custom hooks, and context stores to match the following Javascript object structures:

3.1 Movie Profile Record Shape

JavaScript

```
{
  id: 299534,
  title: "Avengers: Endgame",
  poster_path: "/or0661bY0BvXgC2ttT3Z6G9pYgA.jpg",
  backdrop_path: "/7RyG74xrbfmP3CMILvXmN61I7DY.jpg",
  overview: "After the devastating events of Avengers: Infinity War, the universe is in ruins...",
  release_date: "2019-04-24",
  vote_average: 8.3,
  runtime: 181, // Value populated during deep-detail queries
  genres: [{ id: 28, name: "Action" }, { id: 12, name: "Adventure" }],
  original_language: "en"
}
```

3.2 Cast Credit Listing Shape

JavaScript

{
 id: 299534,
 cast: [
  {
   id: 3223,
   name: "Robert Downey Jr.",
   character: "Tony Stark / Iron Man",
   profile_path: "/1Ykb619zYdbscz00Z09B037g7gA.jpg"
  }
 ]
}

4. System Features / Functional UI Requirements

4.1 FR-1 Home Page Feed Architecture

- **Description:** Loads and segregates separate horizontal movie display matrix tracks onto the entry route.
- **Target API Endpoints:** /trending/movie/day, /movie/popular, /movie/top_rated, /movie/upcoming.
- **Functional Requirements:**
    - Initialize automated network requests during the page mounting phase inside a unified lifecycle container.
    - Maps output arrays into modular card objects displaying poster imagery, text titles, and numeric voting scores.

4.2 FR-2 Movie Search Engine

- **Description:** Filters down matching entries based on user text character inputs.
- **Target API Endpoint:** /search/movie.
- **Functional Requirements:**
    - Capture user keyboard entries via controlled input state mechanisms.
    - Process empty or invalid queries gracefully by updating the DOM view with informative placeholder blocks.

4.3 FR-3 Movie Details Page Component

- **Description:** Renders an informative deep overview board matching an isolated movie asset identifier.
- **Target API Endpoint:** /movie/{movie_id}.

- **Functional Requirements:**
    - Intercept active path variables (:movie_id) using router modules to trigger subsequent individual requests.
    - Bind values for: Poster art, Title, Text summary overview, Genre tags, Runtime minutes, Calendar Release date, Vote rating, and Language track codes .

4.4 FR-4 Cast Details Listing

- **Description:** Lists performance profiles mapping characters to the respective performance actors.
- **Target API Endpoint:** /movie/{movie_id}/credits.
- **Functional Requirements:**
    - Parse response objects to iterate through arrays, outputting the Actor Image, Actor Name, and Character Name inside a grid .

4.5 FR-5 Similar Movies Carousel

- **Description:** Recommends alternative film choices of similar thematic styling or genre metrics.
- **Target API Endpoint:** /movie/{movie_id}/similar.
- **Functional Requirements:**
    - Fetch relational fields matching the current item's view context and append an automated cross-reference recommendation strip below the primary layout view.

4.6 FR-6 Watchlist Memory Manager

- **Description:** Persists selected personal media items directly into client browser memory spaces .
- **Data Cache Target:** Browser LocalStorage utilities.
- **Functional Requirements:**
    - Provide operational add and remove toggle button logic across component cards.
    - Enforce automated serialization string mappings (JSON.stringify, JSON.parse) to synchronize the tracking array across view states without record decay or data loss.

4.7 FR-7 Metadata Filters Sorting

- **Description:** Restricts active list views based on precise filter parameters.
- **Target API Endpoint:** /discover/movie.
- **Functional Requirements:**
    - Expose selection widgets modifying request streams to filter by specified Genre classifications, numeric Rating baselines, and production release Years .

4.8 FR-8 Collection List Pagination

- **Description:** Permits continuous page segment adjustments to load large datasets incrementally.
- **Target Parameters:** Custom ?page={index} query variable modifiers appended across listing routes.
- **Functional Requirements:**
    - Connect interactive index modifier buttons to increment or decrement pagination numbers, updating the endpoint call structure dynamically .

4.9 FR-9 Multi-Device Responsive Design

- **Description:** Enhances visual continuity across varied viewing constraints.
- **Functional Requirements:**
    - Build layouts utilizing defensive CSS strategies to eliminate horizontal scroll bar clipping on Mobile, Tablet, and Desktop displays .

5. Non-Functional Requirements

- **Performance Requirements:** The web client app must perform rendering tasks efficiently, maintaining a target Page Load speed under 3 seconds over standard connection environments.
- **Availability Metrics:** Design for structural reliability, targeting a 99% operational layout uptime framework.
- **Scalability Standards:** Enforce strict atomic component architectural guidelines to simplify future interface additions.
- **Maintainability Metrics:** Separate presentation blocks into single-purpose reusable components to clean up testing and adjustments.
- **Security Constraints:** Sensitive developer production API configuration credentials must remain completely excluded from cleartext scripts, using dedicated environment files instead.

6. Suggested Folder Structure

Students must structure their workspace using the exact structural framework mapped out below:

src/

├── components/

│ ├── Navbar/

│ ├── MovieCard/

│ ├── SearchBar/

│ └── Footer/

├── pages/

```
|   ├── Home/
|   ├── Search/
|   ├── MovieDetails/
|   └── Watchlist/
├── services/
|   └── tmdbApi.js/
├── context/
|   └── WatchlistContext.js/
├── hooks/
|   └── useMovies.js/
├── assets/
├── App.jsx
└── main.jsx
```

7. TMDB API Integration Guide

The client-side request orchestration layer relies on the following structural implementation steps:

- **Step 1: Create Account:** Create a profile inside the TMDB Developer Portal to capture an authorization API Key string .
- **Step 2: Install Axios:** Install the communication client package into your system via npm install axios.
- **Step 3: Create Environment File:** Store credentials locally within .env configuration properties labeled VITE_TMDB_API_KEY=YOUR_API_KEY.
- **Step 4: Create Axios Instance:** Write out the customized service abstraction instance file (src/services/tmdbApi.js):

JavaScript

```
import axios from "axios";


const api = axios.create({
  baseURL: "https://api.themoviedb.org/3",
  params: {
    api_key: import.meta.env.VITE_TMDB_API_KEY
  }
});
```

export default api;

- **Step 5: Fetch Popular Movies:** Expose matching async request functions referencing targeted resource pathways:

JavaScript

```
import api from "./api";


export const getPopularMovies = async () => {
  const response = await api.get("/movie/popular");
  return response.data.results;
};
```

- **Step 6: Use in React:** Bind responses directly into component layouts using localized lifecycle management triggers:

JavaScript

```
const [movies, setMovies] = useState([]);


useEffect(() => {
  getPopularMovies().then(setMovies);
}, []);
```

8. UI Screens Matrix

The application requires four primary presentation screen layouts to handle user interactions:

1. **Screen 1: Home Page:** Features an integrated top Hero Banner layout combined with horizontal browsing strips displaying Trending and Popular categories .
2. **Screen 2: Search Page:** Incorporates a wide text input Search Box aligned above a multi-column responsive Search Results display matrix .
3. **Screen 3: Movie Details Screen:** Renders individual item asset Poster visuals, absolute average ratings, Cast scroll listings, and relative Similar Movie rows .
4. **Screen 4: Watchlist Screen:** Displays a compilation layout collecting all of a user's Saved Movies alongside quick adjustment actions .

9. Bonus Features Framework

Advanced development benchmarks are grouped into four modular feature tiers:

- **Level 1 (+10% Extra Marks):** Integrate a global Context layout theme toggle (Dark/Light Switcher) or an automated Infinite Scroll pagination framework .

- **Level 2 (+15% Extra Marks):** Build an embedded visual trailer modal system executing real-time playback streaming via endpoint requests targeting /movie/{movie_id}/videos .
- **Level 3 (+20% Extra Marks):** Establish a biographical routing layout mapping Actor Details and past performer Filmographies using the path /person/{person_id} .
- **Level 4 (+25% Extra Marks):** Code a custom client-side recommendation engine that processes Genre classifications, user ratings, and active structural logs saved inside the user's Watchlist .

10. Evaluation Rubric Matrix

Performance scoring for student projects follows a strict 100-mark evaluation distribution:

Assessment Component Criterion	**Weightage Maximum**	**Focus Verification Area**
UI Design	10 Marks	Visual alignment, design styling, layout consistency, and typography choices.
React Components	15 Marks	Effective component modularization, state controls, clean property usage, and single-responsibility structures.
Routing	10 Marks	Router path mapping configuration, handling dynamic route matching, and path history controls.
API Integration	20 Marks	Axios initialization setup, handling asynchronous requests lifecycle, and proper error management frameworks.
State Management	15 Marks	Clean usage of Context store engines without causing unnecessary multi-tier re-render events.
Watchlist Feature	10 Marks	Working LocalStorage integration, accurate array update loops, and state synchronization across views.
Responsive Design	10 Marks	Fluid device layout conversions matching Mobile, Tablet, and Desktop monitors cleanly .

Assessment Component Criterion	**Weightage Maximum**	**Focus Verification Area**
Code Quality	5 Marks	Folder structure organization, descriptive system variable names, and clear code comments.
Deployment	5 Marks	Fully compiled application running error-free on remote static cloud servers with hidden API key setups.
TOTAL BASELINE ceiling SCORE	**100 Marks**	Note: Extra credit points from bonus features are added directly onto the final achieved baseline score.

11. 12-Week Detailed Execution & Milestone Plan

The implementation schedule is organized according to the exact weekly deliverable titles defined within the system description text:

Week	**Date**	**Topic**	**What Happens**	**Tools / Concepts**	**Outcome**
1	22/06/2026 To 28/06/2026	Requirement Analysis & TMDB API Study	Define project scope; explore TMDB endpoints (popular, trending, search, etc.), get API key, test calls	Postman, TMDB docs, JSON	Requirements doc + API notes
2	29/06/2026 To 05/07/2026	Wireframe	Sketch rough layouts for home, search, details, and filter pages before writing any code	Figma / Balsamiq / pen & paper	Approved page layouts
3	06/07/2026 To 12/07/2026	Static Design – Part 1	Build the page skeleton: navbar, footer, homepage grid with placeholder movie cards	HTML, Bootstrap grid	Static homepage skeleton
Phase – 1 Evaluation (13/07/2026 to 17/07/2026)					
4	13/07/2026 To 19/07/2026	Static Design – Part 2	Style with custom CSS, finish remaining static	CSS, Bootstrap	Fully styled static site (no live data)

Week	Date	Topic	What Happens	Tools / Concepts	Outcome
			pages (details, search, filters)		
5	20/07/2026 To 26/07/2026	React Setup, Layout & Routing	Scaffold React app, set up routes for home/search/details, build persistent navbar/footer layout	Vite/CRA, react-router-dom	Working navigation between blank pages
6	27/07/2026 To 02/08/2026	Create Components	Break the UI into reusable pieces: MovieCard, MovieGrid, SearchBar, Pagination, CastCard, etc.	React, JSX, props	Component structure (no styling yet)
7	03/08/2026 To 09/08/2026	Design the Components	Style each component to match the wireframe/static design (hover effects, spacing, ratings)	CSS Modules / Bootstrap-in-JSX	Polished, reusable UI components
Phase – 2 Evaluation (10/08/2026 to 14/08/2026)					
8	10/08/2026 To 16/08/2026	Home Page (Live Data)	Fetch & render popular, trending, top rated, upcoming movies; add live search	Axios, TMDB API, useState/useEffect	Fully dynamic, functional homepage
9	17/08/2026 To 23/08/2026	Movie Details Page	On click, show full movie info, cast list, and similar movie suggestions	TMDB credits/similar endpoints, dynamic routes	Working details page
10	24/08/2026 To 30/08/2026	Filter & Pagination	Add genre/year/rating filters and paginate results across multiple pages	TMDB discover endpoint, query params	Complete filter + pagination feature
11	31/08/2026 To 13/09/2026	Responsive Design, Testing	Apply media queries; fix overflow/box rendering issues; audit layout across Mobile, Tablet, Desktop; wrap risky components in React Error Boundaries to	CSS Media Queries, React Error Boundary	Fully responsive, fault-tolerant UI that degrades gracefully on errors/slow connections

Week	Date	Topic	What Happens	Tools / Concepts	Outcome
			handle failed API calls gracefully		
12	14/09/2026 To 20/09/2026	Deployment, Documentation, Presentation	Run production build scripts, secure environment variables, deploy to hosting, prepare final demo	npm run build, Vercel/Netlify/GitHub Pages, .env security	Live deployed app + project documentation & demo ready
Phase – 3 Evaluation (21/09/2026 to 03/10/2026)					

12. Expected Learning Outcomes

Upon successful integration and final presentation assessment, students will be capable of demonstrating technical proficiency across the following development benchmarks:

1. **Production React Scaffolding:** Initializing, designing, and maintaining scalable Single Page Application models using advanced modern developer toolchains.
2. **REST Web Services Consumption:** Setting up custom Axios communications layers to fetch, parse, and render multi-tiered external API datasets.
3. **Application State Management:** Modeling synchronized caching structures that share state parameters across deeply nested component structures.
4. **Client-Side Routing Design:** Implementing dynamic routing frameworks that use parameter and query string tracking to manage page states seamlessly.
5. **Atomic Component Architecture:** Designing independent, single-responsibility visual components that prioritize layout reusability.
6. **Production Cloud Deployments:** Compiling raw source code assets into compressed static production distribution packages hosted on live cloud environments.